

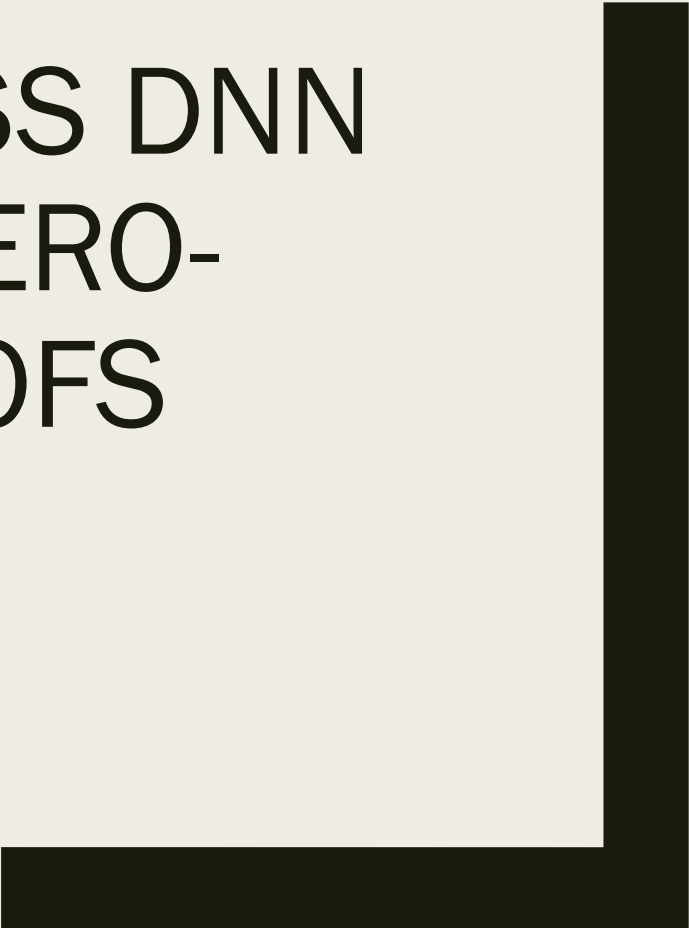


SCALING UP TRUSTLESS DNN INFERENCE WITH ZERO- KNOWLEDGE PROOFS

EE-59903 Advanced Cybersecurity

March 10 2026

Angel Coronel



Trusting ML Services is a problem

- ML-as-a-service
 - Google Vision
 - OpenAI
 - Amazon Rekognition
- Users cannot verify the provider actually ran the model
 - return random predictions
 - run a cheaper model
 - skip computation to save money
 - introduce bugs or malicious modifications





A [Proposed] Solution

- Use ZK proofs to verify NN inference
- Preview contributions
 - zk proofs for ML inference
 - ImageNet-scale neural networks
 - applications in ML verification



Zero-Knowledge Proofs

- A cryptographic protocol where
 - A prover convinces a verifier that a statement is true without revealing any information beyond the truth of the statement
 - Formally, a proof system is zero-knowledge if the verifier learns nothing except that the statement is valid
- The verifier learns no information beyond the validity of the statement
 - There exists a simulator that can generate transcripts indistinguishable from real ones
 - This means the verifier could have generated the proof themselves, so they learned nothing

ZK Definition (informal)

■ Completeness

- *If a statement is true and both the prover and verifier follow the protocol honestly, the verifier will be convinced of the statement's truth*
- *Proof system is comprehensive so that it can prove all true statements*

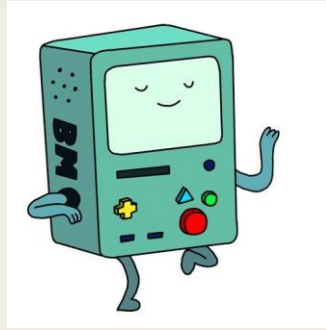
■ Soundness

- *If a statement is false, no dishonest prover can convince an honest verifier that it is true, except with a negligible probability*
- *Whatever is proven, it is true*

■ Zero knowledge

- *Proof reveals no private info*

Prover



knows secret y
s.t. $y^2 \equiv x \pmod{n}$

public input: $(x, n) \rightarrow$

1. Choose random r
2. Compute $u = r^2 \pmod{n}$

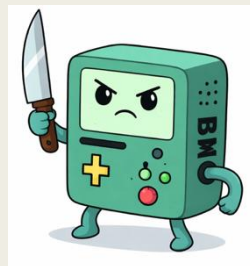
$u \rightarrow$

$\leftarrow$ challenge b

4. If $b = 0$ then send r
If $b = 1$ then send $r*y \pmod{n}$

$r \mid r*y \pmod{n} \rightarrow$

Repeat protocol k times and
cheating probability $\leq 2^{-k}$

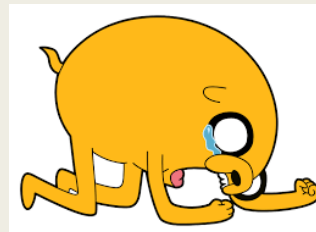
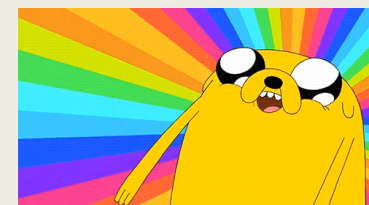


Verifier



3. Pick random bit $b \in \{0,1\}$

5. Verify:
If $b = 0$: check $r^2 \equiv u \pmod{n}$
If $b = 1$: check $(r*y)^2 \equiv u*x \pmod{n}$
accept / reject



From ZK Proofs to ZK-SNARKs

- Original ZK proofs:
 - *interactive*
 - *large proofs*
 - *inefficient*
- ZK-SNARKs
 - *Succinct*
 - Proofs are small
 - *non-interactive*
 - verifier doesn't need to participate
 - *fast verification*
 - *ZK*
 - hidden data stays secret
 - *Soundness*
 - cheating provers cannot fake proofs



Verifiable Computation

General SNARK idea:

- Prove:

- $y = f(x)$

- without revealing:

- x

- *internal computation*

Note: x is a public
parameter in original ZK

Applying This to Neural Networks

- Want to prove $\text{model}(\text{input}) = \text{output}$
 - *without revealing: model weights & input data*
- But neural networks contain:
 - *floating point math*
 - *nonlinear functions*
 - *millions of operations*



Converting Neural Networks to SNARK Circuits

- Change neural network to arithmetic circuit
- Operations:
 - *additions*
 - *multiplications*
 - *Constraints*
- Automatic Circuit Generation
 - *Make the TensorFlow Lite model turn into a SNARK circuit*
 - *Optimizations:*
 - circuit reuse
 - shared lookup tables

Techniques

Quantized Neural Networks

- Floating-point math is expensive
- Convert network to 8-bit integers
 - *smaller circuits*
 - *faster proofs*

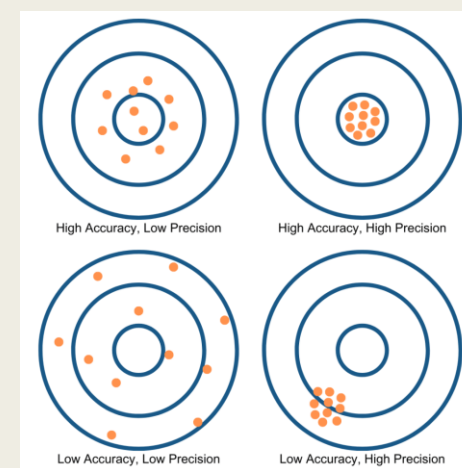
Custom Gates

- Neural network layers become **dot products**
- Example:
 - $c = \sum(x * w)$
- Custom gates enforce this in the SNARK circuit

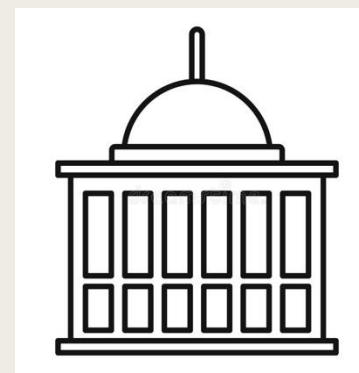
Lookup Tables

- Activation functions like ReLU are nonlinear
- Instead use lookup tables
 - *check that (input, output) $\in$ table*

Applications of Verified ML



- verify model accuracy
 - *If company claims model is 80% accurate, consumer **knows the real accuracy** without seeing the weights*
- verify ML-as-a-service predictions
 - *Instead of trusting provider, provider sends prediction + ZK proof and you can verify proof confirms prediction came from the model*
- trustless document retrieval
 - *A company could hide documents matching some AI classifier. A requestor (like an audit from government) can verify no documents were hidden*



Experimental Results

- Verification is very fast (under ~10 seconds)
- Proof sizes are tiny (~6–7 KB)
- Proving time grows with model size

Scaling up Trustless DNN Inference with Zero-Knowledge Proofs

Model	Accuracy (top-5)	Setup time	Proving time	Verification time	Proof size (bytes)
MobileNet, 0.35, 96	59.1%	93.9s	163.2s	0.74s	6528
MobileNet, 0.5, 224	75.7%	937.7s	1530.7s	6.32s	7552
MobileNet, 0.75, 192	79.2%	1341.2s	2457.5s	10.27s	5952

Comparison with Prior Work

- Previous zkML systems:
 - *slower*
 - *limited to small models*
- This paper:
 - *first ImageNet-scale zk inference*

Method	Proving time lower bounds (s)
Zen	20,000
vCNN	172,800
pvCNN	31,011*
zkCNN	1,597*

Strengths

- first large-scale zkML system
- practical verification times
- strong cryptographic guarantees

Limitations

- proving time expensive
- limited model architectures
- still not practical for very large models

Discussion - Do we actually need verifiable ML?

- What technical improvements would be necessary before verified ML could be used in real-time ML services?
- If so, where?
 - *cloud AI services*
 - *regulatory environments (finance, healthcare)*
 - *auditing AI systems*
 - *safety-critical systems*
- Where do we NOT need it?
 - Asking “What can I cook for dinner today?” to an LLM does *not need* to be cryptographically verified
- Would companies realistically generate proofs for every inference?
- Are there industries where verification would be mandatory? Should be mandatory?

Discussion – Is it practical?

- Is verified ML inference practical today, or is it still mainly a research prototype? Can zkML Scale to Modern AI Models?
 - Current top models range in parameters from billions to trillions
- Would verifying LLM inference even be useful?
 - *If so, what types of applications could tolerate the long proving time required by ZK-SNARK verification?*
- How much accuracy loss is acceptable when adding cryptographic verification to ML models?
- Should verification systems prioritize correctness guarantees or predictive performance?
- Could future hardware or cryptographic improvements remove the need for these approximations?

Scaling top models vs paper model

Proof Times

Model	Relative Size	Estimated Proving Time
MobileNet v2	1×	minutes
Gemini (100B)	~30k×	months
DeepSeek R1	~200k×	years
GPT-4	~500k×	multiple years
GPT-5	~1M×	decades

Proof Size

Metric	MobileNet	LLM
Proof size	~6 KB	~6–20 KB
Verification time	seconds	seconds
Proving time	minutes	months/years

Discussion – Criticizing the paper

- Who is the intended audience of this paper: ML researchers, cryptographers, or both?
- How could the authors make the paper more accessible to readers outside the zero-knowledge proof community?
- What background knowledge is necessary to fully understand the paper?

Citations

Shafi Goldwasser, Silvio Micali, and Charles Rackoff. *The Knowledge Complexity of Interactive Proof Systems*. Proceedings of the 17th Annual ACM Symposium on Theory of Computing (STOC), 1985.

Srinath Setty. *Scaling up Trustless DNN Inference with Zero-Knowledge Proofs*. Proceedings of the 37th International Conference on Machine Learning (ICML), 2020.

Cleve's Impossibility

- Two parties A, B
- Protocol Π
- r rounds
- ϵ -consistency: $\Pr[\Omega_a = \Omega_b] \geq \frac{1}{2} + \epsilon$
- δ -Bias of Ω_a : $\left| \Pr[\Omega_a = 1] - \frac{1}{2} \right| \geq \delta$
- At most one corrupt party

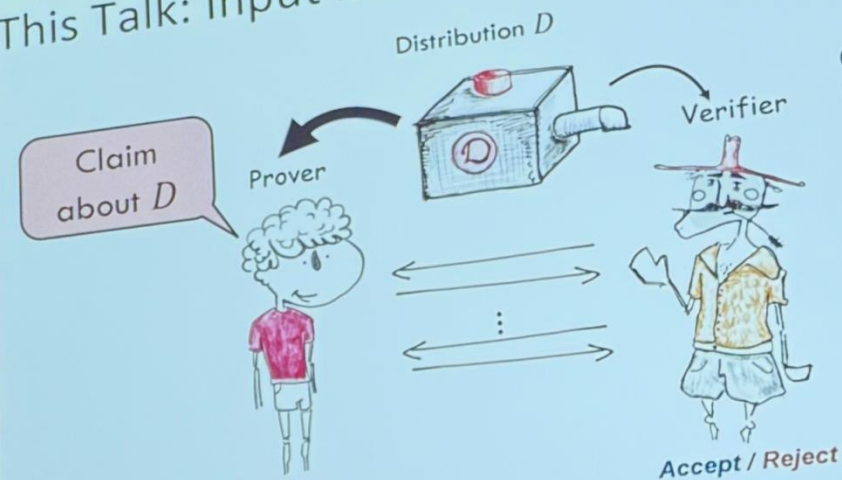


Output: $\Omega_a, \Omega_b \in \{0,1\}$



Cleve's result: A single corrupt party in an ϵ -consistent protocol can cause $\frac{\epsilon}{4r+2}$ bias.

This Talk: Input is Samplable Distribution

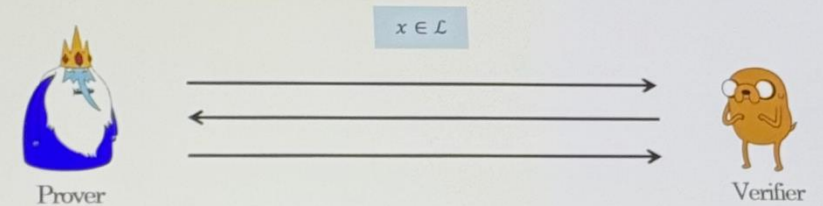


Completeness
If D satisfies claim, V accepts whp.

Soundness
If D is far from satisfying claim, for any cheating prover V rejects whp.

My experience with crypto presentations

Zero Knowledge Proofs [Goldwasser-Micali-Rackoff'89]



Zero-Knowledge:

There is a **poly-time simulator** for the interaction between an honest prover and *any* verifier.

